

Assignment 3

Database Management Systems — Weeks 5 and 6

Part I: Concepts & Theory (Short Answer)

Q1. The Four ACID Properties

Atomicity – A transaction is treated as a single indivisible unit: either all of its operations succeed, or none of them do. Example: in a fund transfer, if the debit from one account succeeds but the credit to the other fails, the entire transaction is rolled back so no money simply vanishes.

Consistency – A transaction moves the database from one valid state to another, respecting all rules, constraints, and relationships defined on the data. Example: if a rule states that account balances can never go negative, a transaction that would violate this is not allowed to complete.

Isolation – Concurrently running transactions should not interfere with each other; the outcome should look as if transactions were executed one after another. Example: while one customer is booking the last seat on a flight, another customer attempting to book the same seat should not be able to see or grab it until the first transaction finishes.

Durability – Once a transaction is committed, its effects are permanent, even if the system crashes immediately afterward. Example: after an ATM withdrawal is confirmed and the receipt is printed, the updated balance survives even if the bank's server loses power a second later.

Q2. Crash Between UPDATE and COMMIT

If the database crashes after the first UPDATE (the deduction of ₹500 from Account 101) but before COMMIT is reached, the transaction is left incomplete. Because the transaction was never committed, the database's recovery process treats it as an uncommitted transaction and automatically rolls it back when the system restarts. This means Account 101's balance is restored to its original value, and the ₹500 is never actually lost — Atomicity guarantees that a half-finished transfer cannot remain in the database permanently.

COMMIT is used when all the statements inside a transaction have executed successfully and the changes should be made permanent and visible to other users. ROLLBACK is used when something goes wrong partway through — an error, a failed constraint check, or an explicit decision by the application — and the goal is to undo every change made since the transaction began, returning the database to the state it was in before the transaction started.

Q3. Concurrency Phenomenon

This is a classic example of a non-repeatable read: User A reads the same row twice within one transaction and gets two different values because User B modified and committed a change to that row in between the two reads.

The ACID property responsible for preventing this is Isolation, and the transaction concept used to control it is the isolation level (for example, using Repeatable Read or Serializable isolation, or applying

appropriate locking, so that a transaction's own reads stay consistent for the duration of that transaction).

Q4. Views in SQL

A View is a virtual table defined by a stored SQL query. It does not store data of its own; instead, it presents data from one or more underlying tables and is recomputed whenever it is queried.

Two advantages of using a view:

- Simplicity and reuse — complex joins or calculations can be written once inside the view, so users can query the view with a simple SELECT instead of repeating the complicated logic every time.
- Security — a view can expose only specific columns or rows of a sensitive table (for example, hiding a Salary column from most users), so it acts as a restricted window into the data.

Data cannot always be updated directly through a view. A view is generally updatable only if it satisfies conditions such as: it is based on a single base table (no joins), it does not use GROUP BY, HAVING, DISTINCT, or aggregate functions (SUM, COUNT, AVG, etc.), it does not use UNION or set operations, and every column not included in the view either allows NULLs or has a default value so that INSERT statements through the view can still succeed. If any of these conditions are violated, the view is read-only.

Q5. Common Table Expressions (CTEs)

A Common Table Expression (CTE) is a temporary, named result set defined using a WITH clause that exists only for the duration of the single query that follows it. Unlike a normal subquery, which is nested inline inside a FROM or WHERE clause and can quickly become hard to read, a CTE is defined upfront and referenced by name afterward, which makes the overall query more readable and lets the same CTE be referenced multiple times within the query without rewriting it.

A non-recursive CTE runs once and simply returns its result set, acting mainly as a way to organize or simplify a query. A recursive CTE, on the other hand, refers to itself and repeats execution, building on its own previous output until a stopping condition is met — it consists of an initial (anchor) query combined with a recursive query using UNION ALL. A common use case for a recursive CTE is traversing hierarchical data, such as generating an organizational chart of a manager and all employees who report to them (directly or indirectly).

Q6. Stored Procedures

A Stored Procedure is a precompiled, named group of one or more SQL statements that is stored inside the database itself and can be executed (called) as a single unit whenever needed, optionally accepting input parameters and returning output values.

Two primary advantages over standard individual queries:

- Performance — because the procedure is precompiled and its execution plan is cached by the database engine, it typically runs faster than sending the same set of raw queries from the application each time.

- Maintainability and security — business logic lives in one central place inside the database rather than being duplicated across application code, and access can be granted to execute the procedure without giving direct access to the underlying tables, reducing the risk of SQL injection and unauthorized data access.

Part II: Practical SQL Application

Reference table used for Q7, Q8, and Q9:

EmpID	Name	Department	Salary
1	Alice	HR	60000
2	Bob	Sales	90000
3	Carol	Sales	85000
4	David	HR	75000
5	Eve	IT	95000

Q7. Create the SalesEmployees View

```
CREATE VIEW SalesEmployees AS
SELECT EmpID, Name, Salary
FROM Employees
WHERE Department = 'Sales';
```

Q8. Transaction to Update Eve's Salary

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = Salary + 5000
WHERE Name = 'Eve';

COMMIT;
```

Q9. CTE for Above-Average Salaries

```
WITH AvgSalaryCTE AS (
    SELECT AVG(Salary) AS AvgSalary
    FROM Employees
)
SELECT e.Name, e.Salary
FROM Employees e, AvgSalaryCTE a
WHERE e.Salary > a.AvgSalary;
```

Part III: Long Answer / System Design

Q10. Online Shopping Platform — Order Processing

A. Why these four operations must be a single transaction

These four operations — deducting inventory, recording payment, creating the order entry, and updating order history — are all different facets of one logical business event: "a customer placed an order." If they were run as independent queries, there would be no guarantee that either all four happen or none do. A failure between steps (a network glitch, a server crash, an application bug) could leave the system in an inconsistent state, for example charging a customer without ever creating their order, or reserving inventory that is never actually paid for. Wrapping them in a single transaction ensures the entire order either completes fully and correctly, or is undone completely, keeping the data trustworthy.

B. How each ACID property ensures correctness here

- **Atomicity:** guarantees all four steps succeed together or are rolled back together, so there is never a state where, say, payment is recorded but no order exists.
- **Consistency:** ensures business rules stay valid throughout, for instance that inventory can never go negative and that a payment record always corresponds to a real order.
- **Isolation:** ensures that when thousands of orders are being placed per minute, one customer's order does not read or overwrite half-finished data from another customer's simultaneous order, particularly around shared inventory counts.
- **Durability:** ensures that once the platform confirms the order to the customer, that confirmation is permanent — the order, payment, and inventory update survive even if the server crashes moments later.

C. Real-world problem if Atomicity were not enforced

Without Atomicity, it is entirely possible for the payment to be successfully recorded and the customer's card charged, while the order entry creation or inventory deduction step fails afterward due to a crash or error. The customer would then be charged money but would have no order in the system and no product shipped — a direct financial and trust problem for both the customer and the business, requiring manual refunds and support intervention.

D. Purpose of Isolation and a concurrency risk without it

On a high-traffic platform, many customers may attempt to buy the same product at nearly the same instant. The purpose of Isolation is to make sure each order transaction sees a consistent, correct view of shared data such as stock levels, and that one transaction's in-progress changes are not visible to (or overwritten by) another transaction until it is safely committed.

A concrete risk without proper isolation is a Dirty Read leading to overselling: suppose only one unit of an item is left in stock. Transaction A reads the stock as 1 and begins deducting it but has not yet committed. If Transaction B is allowed to read that same uncommitted value or race in before A commits, B might also see 1 unit available and proceed to sell it too. Both transactions could end up believing the sale succeeded, so the platform sells the same last unit twice — resulting in negative or incorrect inventory and an order that can never actually be fulfilled. Using an appropriate isolation level

(such as Read Committed with row-level locking, or Serializable for the critical stock-check-and-deduct step) prevents this by ensuring inventory reads and writes for the same item are properly serialized.